

Retreinamento Reativo em Aprendizado Federado sob Concept Drift Gradual: Detecção Local por Incerteza e Decisão Coletiva por Quórum

Julio Vinicius Amaral Oliveira*, Hugo César de Oliveira†, Miguel Elias M. Campista†, Luiz Fernando Bittencourt*

*Instituto de Computação, Universidade Estadual de Campinas (UNICAMP), Brasil

j230537@dac.unicamp.br, bit@unicamp.br

†Universidade Federal do Rio de Janeiro (UFRJ), Brasil

hugocesar@gta.ufrj.br, miguel@gta.ufrj.br

Resumo—Modelos de Aprendizado Federado (Federated Learning – FL) empregados em ambientes reais enfrentam desvio de conceito (concept drift), que se refere à mudança da distribuição dos dados após a implantação do modelo, levando à deterioração da qualidade da aplicação correspondente até que seja adotada uma ação de retreinamento. Em cenários reais, a mudança nem sempre é abrupta; a fração de entradas degradadas cresce lentamente ao longo de um intervalo de transição, cuja duração é a largura do drift. Este trabalho propõe uma política de decisão coletiva por quórum para disparar o retreinamento reativo sob drift gradual. Cada cliente monitora um fluxo de imagens não rotuladas com um detector local baseado em incerteza, que emite um alarme ao identificar mudança na distribuição. O servidor acompanha os alarmes a cada passo de monitoramento e mantém, ao longo dos passos recentes, o registro dos clientes distintos que sinalizaram mudança. O retreinamento sobre os dados correntes somente é disparado quando a fração desses clientes atinge um quórum mínimo, ainda que os alarmes venham de passos diferentes. A regra exige confirmação de clientes distintos, de modo que um falso positivo local isolado não basta, e acumula alarmes espalhados no tempo, o que é decisivo quando clientes diferentes sofrem drift em momentos diferentes (drift staggered). A avaliação usa imagens CIFAR-10 sob quatro degradações visuais, com cinco sementes e três durações de rampa (50–200s), totalizando 60 episódios que comparam a variante reativa com uma referência sem retreinamento. As métricas são a indisponibilidade de serviço, calculada a partir do tempo acumulado em que a acurácia do modelo em serviço permanece abaixo de um limiar de qualidade, e o atraso de detecção. A política detecta o drift em 59 de 60 episódios e reduz o tempo médio de indisponibilidade de 408–562s para 0–178s, anulando-o no melhor caso; rampas longas também preservam melhor a acurácia no conjunto limpo. O gatilho coletivo não dispara em 15 controles sem drift (zero falsos positivos) e é acionado em todos os episódios de drift staggered, enquanto o ruído Gaussiano mostra-se o cenário mais difícil. O trabalho contribui ainda com a métrica de indisponibilidade de serviço, um detector de incerteza federado e uma infraestrutura experimental auditável. No regime avaliado, uma política reativa sem rótulos é eficaz, e a latência de detecção depende do cenário.

Palavras-chave—aprendizado federado, concept drift, detecção de drift, retreinamento reativo, decisão coletiva por quórum, detecção não supervisionada, indisponibilidade de serviço

I. Introdução

O Aprendizado Federado (Federated Learning – FL) treina um modelo compartilhado a partir de dados distribuídos entre os clientes para preservar a privacidade [1]. Uma vez implantado, o modelo é servido em um ambiente cuja distribuição dos dados dos clientes pode mudar; tal concept drift compromete a qualidade do modelo em serviço até que uma ação de retreinamento seja adotada. Detectar mudança nos dados, em geral sem rótulos no momento da inferência, é um pré-requisito para dispará-la. A mudança nem sempre é abrupta. Por exemplo, no drift gradual, dados novos passam a coexistir com os antigos, e a proporção de entradas deslocadas cresce lentamente ao longo de um intervalo de transição cuja duração é a largura do drift, definição importada da literatura não federada de detecção de drift [3], [18].

A política reativa adotada segue a arquitetura predominante na literatura de drift em FL, de detecção local e decisão central [2], [4]–[10], [24]. A detecção local é a resposta conhecida à falha dos testes agregados no servidor sob drift staggered (drift escalonado entre clientes) [4]. A contribuição deste trabalho está principalmente na regra que converte o sinal local em disparo de retreinamento. Nos trabalhos anteriores, o que se faz com o alarme varia (mitigação no próprio cliente, o endpoint [5], [6], reagrupamento de clientes [9], [10] ou controle de participação [7]) e não exige a confirmação de clientes distintos ao longo do tempo. O caso mais próximo, a barreira de participação do FL-MalDrift [7], regula a participação de clientes, não o disparo.

Diferentemente da literatura, este trabalho propõe uma política de decisão coletiva por quórum, em que o servidor somente dispara o retreinamento quando uma fração mínima de clientes distintos sinaliza alarme dentro de uma janela de passos, o que (i) impede que um falso positivo local isolado sob monitoramento contínuo [18] baste para iniciar o retreinamento e (ii) acumula alarmes de clientes que sofrem drift em momentos diferentes. Na detecção, cada cliente executa o detector de drift por

incerteza (uncertainty-based drift detection – UDD) [11] sobre o modelo global compartilhado, em vez da configuração centralizada original, na qual o Monte Carlo (MC) dropout [12] produz previsões estocásticas em T forward passes, a entropia média de Shannon resume a incerteza dessas previsões e um teste de janela adaptativa (Adaptive Windowing – ADWIN) [13] monitora essa série para sinalizar a mudança. Uma vez disparado, o retreinamento é uma única rodada global sobre os dados que cada cliente serve no momento, em consonância com os orçamentos explícitos de retreinamento do DriftGuard [8] e do ShiftEx [26] e com a separação entre gatilho e seleção de dados do Modyn [20]. Este trabalho modela o concept drift gradual como uma rampa linear da fração de dados degradados, variando de zero a um ao longo de um número configurável de passos de monitoramento [3]. Para avaliar a proposta, utiliza-se como métrica primária a indisponibilidade do serviço (service downtime), ou seja, o tempo acumulado em que a acurácia do modelo em serviço sob degradação permanece abaixo de um limiar de qualidade τ . Os experimentos abrangem rampas curtas (o regime abrupto) e uma rampa longa, na qual o detector opera em meio à transição. Até onde se sabe, nenhum trabalho na literatura revisada sobre drift em FL quantifica o tempo em que um modelo em serviço permanece abaixo de um limiar de qualidade. Dessa forma, as principais contribuições deste trabalho são:

- uma política de decisão coletiva por quórum que agrega alarmes sem rótulos por cliente em uma janela de passos e exige a confirmação de clientes distintos antes de disparar o retreinamento. Essa política está ausente na literatura revisada; a mais próxima é a barreira de participação do FL-MalDrift [7];
- caracterização da latência de detecção por meio de análises de sensibilidade ao quórum e à janela, além de experimentos sob drift staggered;
- uma métrica de indisponibilidade de serviço para FL sob drift, que acumula o tempo em que a acurácia do modelo em serviço permanece abaixo de um limiar de qualidade τ ;
- uma implementação federada do UDD com um detector por cliente compartilhando o modelo global, estendendo o detector centralizado de Baier et al. [11];
- uma infraestrutura experimental auditável que empareja a variante reativa com uma abordagem de referência sem retreinamento a partir do mesmo checkpoint, estado aleatório, fluxo e horizonte; e
- uma caracterização empírica do drift gradual ao longo de 60 episódios pareados.

II. Contextualização e Trabalhos Relacionados

A. Concept drift em aprendizado federado

O concept drift é a mudança na distribuição dos dados ao longo do tempo [19], [21]. Com entradas X e rótulos Y , a taxonomia distingue drift virtual (muda apenas a distribuição das entradas, $P(X)$) de drift real (muda a

condicional dos rótulos dada a entrada, $P(Y | X)$), bem como drifts abruptos, graduais e recorrentes [19]. Os staggered drifts, nos quais clientes diferentes sofrem drift em momentos diferentes, são estudados por Jothimurugesan et al. [4]. O drift gradual, regime estudado neste trabalho, combina o conceito novo com o antigo em proporção crescente ao longo do tempo [2]. A duração da transição é a largura do drift, definida na literatura de detecção não federada [3], [18].

O escopo deste trabalho limita-se a drifts acompanhados de mudança em $P(X)$ (covariate shift, com rótulos preservados). É o pressuposto sob o qual a detecção sem rótulos é estruturalmente possível, pois métodos sem rótulos são funções determinísticas das entradas, de modo que uma mudança restrita a $P(Y | X)$ não produz sinal observável, uma limitação relatada por Solozobov [17] e afirmada por Baier et al. [11].

B. Detecção local, decisão central

Um achado recorrente é que a detecção de drift precisa ser local. Jothimurugesan et al. [4] mostram que testar o erro agregado no servidor falha durante transições staggered, porque os erros de clientes que já sofreram drift e dos que ainda não sofreram se cancelam. FLARE [5], FLAME [6], FL-MalDrift [7], DriftGuard [8], FedDAA [9], FedPLC [10], FELK/FL-HVD [24] e Casado et al. [2] seguem detecção local com decisão central. Em outros trabalhos, o FLASH detecta drift no servidor a partir da magnitude das atualizações agregadas de parâmetros [23]. Stallmann et al. usam um detector global por agrupamento fuzzy [25] e o Adaptive-FedAVG elimina por completo a detecção, adaptando a taxa de aprendizado no servidor para aumentar a plasticidade da fase de aprendizado [27].

Este trabalho adota a arquitetura predominante porque a detecção local é a resposta da literatura à falha dos testes agregados no servidor. Porém, a contribuição deste trabalho não está no sinal local, mas na regra que o transforma em decisão. Nesse ponto, o que se faz com o alarme varia entre os trabalhos revisados e, em geral, exige menos confirmação: FLARE e FLAME acionam a mitigação a partir do próprio endpoint [5], [6]; FedDAA e FedPLC reorganizam os clientes por agrupamento [9], [10]; o DriftGuard formaliza no servidor a tupla gatilho/participação/parâmetros [8]; e o FL-MalDrift usa uma barreira de participação que regula quais clientes agregam, não quando disparam [7]. Em nenhum desses trabalhos, a decisão exige a confirmação de múltiplos clientes distintos ao longo de uma janela temporal. Neste trabalho, o disparo requer um quórum de clientes distintos que sinalizaram, possivelmente em passos diferentes da janela, com as duas propriedades já enumeradas na Seção I.

C. Detecção de drift sem rótulos

O UDD de Baier et al. [11] é o método de referência sem rótulos. O ADWIN é preferido aos detectores DDM e EDDM (Drift Detection Method e Early Drift Detection

Method), porque estes exigem entradas binomiais, enquanto o sinal de entropia é contínuo [11]. Lobo et al. [14] conectam concept drift e incerteza do modelo, e sabe-se que estimadores de incerteza perdem confiabilidade sob dataset shift [15]. O D3M [16] é um método complementar sem rótulos, baseado em discordância entre modelos, com garantias quanto a falsos positivos sob o pressuposto de covariate shift. O PUDD [28] é igualmente complementar, pois aplica um teste qui-quadrado de Pearson a um índice de incerteza de predição, cujo nível de significância controla a taxa de falsos positivos, ao contrário da etapa ADWIN do UDD, que acompanha a média.

D. Orquestração e orçamentos de retreinamento

O DriftGuard [8] formaliza a decisão de retreinamento no servidor em três componentes: o gatilho (se deve retreinar), a participação (quais dispositivos participam) e os parâmetros (quais devem ser atualizados). O trabalho relata reduções de custo de retreinamento de 80–83%. O Modyn [20] separa políticas de gatilho (tempo, volume, desempenho, drift) de políticas de seleção de dados. Seu gatilho de drift usa a máxima discrepância média (Maximum Mean Discrepancy – MMD), com o teste de Kolmogorov-Smirnov (KS) entre as estatísticas candidatas, e um limiar adaptativo por percentil. Nos trabalhos revisados de FL, somente o DriftGuard [8] e o ShiftEx [26] explicitam o orçamento de retreinamento (a quantidade que limita o número de rodadas após o gatilho). Este trabalho adota um orçamento de uma rodada sobre a distribuição que cada cliente serve no momento.

E. Lacunas de métrica e de decisão

O benchmark de detecção de drift de Cerqueira et al. [3] documenta a falta de métricas temporais padronizadas (atraso normalizado de detecção, taxa de alarmes, janelas de aceitação). Há também levantamentos de drift em FL [19]. A latência de detecção [5], [11], a duração do retreinamento e o custo por retreinamento [8] são relatados separadamente, e nenhum dos trabalhos conhecidos quantifica, em uma métrica de indisponibilidade, o tempo em que o modelo em serviço permanece abaixo de um limiar de qualidade, à semelhança das práticas de objetivos de nível de serviço (Service Level Objective – SLO) em MLOps (Machine Learning Operations). A linguagem de indisponibilidade existe em trabalhos adjacentes de sistemas de aprendizado de máquina. O desaprendizado (unlearning) federado, por exemplo, mede a indisponibilidade do serviço como o tempo de inatividade incorrido ao retreinar para apagar dados [29]. Essa noção de disponibilidade (serviço fora de operação) é distinta do tempo de operação abaixo do limiar de qualidade usado aqui.

Além da métrica, nenhum dos trabalhos revisados condiciona o disparo do retreinamento à confirmação de clientes distintos ao longo de uma janela temporal. Este trabalho

propõe uma política de decisão coletiva por quórum que exige essa confirmação.

III. Definição do Problema

Considera-se o FL síncrono com C clientes e tempo v , o relógio abstrato desta infraestrutura experimental. Um modelo é treinado em dados limpos durante R rodadas e entra em operação no instante v_0 . Os dados de inferência chegam em passos de monitoramento de Δt segundos, um lote sem rótulos por cliente a cada passo; o passo t encerra-se no instante $v = v_0 + t \Delta t$.

Modelo de drift gradual. Uma degradação aplicada às entradas X (rótulos preservados) é injetada com uma fração degradada $f(v)$ que cresce linearmente de zero a um ao longo de N passos:

$$f(v) = \min\left(1, \frac{v - v_0}{N \Delta t}\right), \quad v \geq v_0. \quad (1)$$

A duração da rampa $N \Delta t$ é a largura do drift; sua inclinação $1/N$ por passo é a variável independente. Aqui $N \in \{5, 10, 20\}$ (50, 100, 200 s com $\Delta t = 10$ s).

Limiar de qualidade e downtime. Seja $a(v)$ a acurácia do modelo em serviço em um conjunto de teste com a fração corrente $f(v)$. O limiar de qualidade τ define o estado do serviço; o modelo está indisponível quando $a(v) < \tau$. Como a acurácia é avaliada apenas em instantes discretos, o downtime ao longo de $[v_0, v_0 + H]$ é a soma da duração de cada intervalo entre avaliações consecutivas, sempre que a acurácia medida no início do intervalo é inferior a τ . Formalmente,

$$D = \sum_{i=1}^m (v_{i+1} - v_i) \cdot \mathbf{1}[a(v_i) < \tau], \quad (2)$$

em que $v_1, \dots, v_m$ são os instantes de avaliação e $v_{m+1} = v_0 + H$. A acurácia mantém o último valor medido até a próxima avaliação e, no intervalo final, até o horizonte $v_0 + H$. Essas avaliações ocorrem nas fronteiras dos passos de monitoramento (a cada Δt segundos), no instante em que o retreinamento termina (uma avaliação extra que registra a recuperação logo após a rodada, mesmo que o término caia entre duas fronteiras) e no horizonte $v_0 + H$.

IV. Método Proposto

A política reativa combina treino limpo, monitoramento local sem rótulos e uma decisão coletiva por quórum, que dispara o retreinamento quando os alarmes atingem o quórum mínimo. O Algoritmo 1 resume o fluxo; os parâmetros estão na Seção V.

A. Detecção local por incerteza (UDD por cliente)

Cada cliente c executa um detector local sobre o modelo global compartilhado: (1) o modelo é posto em modo de avaliação com as camadas de dropout reativadas; (2) T passagens forward estocásticas produzem probabilidades por classe; (3) calcula-se a entropia média de Shannon $\mathcal{H} = -\sum_k p_k \log p_k$ sobre o lote; (4) o escalar é usado

como entrada de um ADWIN por cliente com sensibilidade δ [11], [13]. O ADWIN compara a média de duas subjanelas do fluxo e somente sinaliza quando há observações suficientes para a comparação. Como cada cliente fornece uma única observação por passo (a entropia média do lote), esse histórico mínimo exige vários passos. Por isso, cada detector passa por passos iniciais de warm-up (calibração) em dados limpos antes da operação, e os alarmes desse período não entram na decisão coletiva.

B. Decisão coletiva por quórum

A cada passo de operação, o servidor recebe os alarmes por cliente. Seja S_t o conjunto de clientes que emite alarme no passo t ; a política mantém a união dos alarmes na janela dos últimos W passos, $\hat{S}_t = \bigcup_{i=t-W+1}^t S_i$, e dispara o retreinamento no passo t_d , quando $|\hat{S}_{t_d}|/C \geq \theta$; o instante de relógio do disparo é $v_d = v_0 + t_d \Delta t$. Dois parâmetros definem a política: o quórum θ , que controla quanta confirmação se exige para agir, e a janela W , que controla por quanto tempo alarmes de clientes distintos podem se acumular. Desse modo, um falso positivo local isolado sob monitoramento contínuo [18] não basta para iniciar o retreinamento. Após o disparo, a política não decide novamente no episódio, de modo que cada episódio tem no máximo um retreinamento.

C. Retreinamento reativo sobre os dados correntes

No instante v_d , o conjunto de treinamento de cada cliente é substituído por um conjunto com a fração degradada do instante do disparo, $f(v_d)$ (Eq. (1)), e um orçamento de K rodadas síncronas de FL é executado com todos os clientes. Cada cliente detém uma fração IID (dados independentes e identicamente distribuídos) do conjunto de treinamento de n_{tr} imagens (n_{tr}/C imagens), na fração degradada corrente.

D. Avaliação sobre os dados correntes

A série de acurácia é medida no conjunto de teste de n_{te} imagens com a fração corrente $f(v)$; no início o

Algoritmo 1 Retreinamento reativo com detecção local por incerteza e decisão coletiva por quórum.

```

1: Entrada:  $C$  clientes;  $R$  rodadas limpas;  $T$  passagens com MC
   dropout; sensibilidade  $\delta$ ; quórum  $\theta$ ; janela  $W$ ; orçamento de
   retreinamento  $K$ ; treino  $n_{tr}$ ; teste  $n_{te}$ 
2: servidor treina o modelo global ( $R$  rodadas); cada cliente inicializa
   e calibra seu detector com dados limpos
3: enquanto o serviço está em operação faça
4:   para cada cliente  $c$  em paralelo faça
5:     recebe um lote sem rótulos e estima a entropia média com
       MC dropout ( $T$  passagens); emite alarme se o ADWIN sinaliza
       mudança
6:   fim para
7:   se a fração de clientes distintos com alarme na janela  $W$  atinge
       o quórum  $\theta$  então
8:     dispara  $K$  rodadas de retreinamento global sobre os dados
       correntes
9:   fim se
10: fim enquanto

```

Tabela I
Calibração de severidade a partir do checkpoint limpo (intervalo estrito (0,25; 0,50)).

Degradação	Severidade	Acurácia
Ruído Gaussiano	3	0,3783
Frosted-glass blur	4	0,4022
Motion blur	1	0,3493
Fog	4	0,4875

teste é limpo e somente se torna totalmente degradado quando a rampa termina. A escolha de quais imagens são degradadas segue uma permutação determinística, de modo que as duas variantes avaliam exatamente as mesmas imagens em cada instante.

E. Calibração de severidade

Cada degradação é parametrizada por uma severidade inteira e crescente, em que valores maiores correspondem a perturbações mais fortes. A calibração parte de um checkpoint limpo e avalia todas as severidades no conjunto de teste degradado; seleciona-se a menor severidade cuja acurácia pertence a uma faixa-alvo. O limite inferior mantém o modelo acima do acaso, de modo que a recuperação é alcançável com orçamento reduzido de retreinamento, e o limite superior garante que a degradação leve a acurácia do serviço abaixo de τ .

V. Configuração Experimental

A. Conjunto de dados e degradações

Emprega-se CIFAR-10 com partição IID entre 10 clientes, com as 10 classes completas em todas as fases. Quatro degradações visuais são implementadas diretamente sobre tensores (ruído Gaussiano, frosted-glass blur, motion blur e fog), com severidades inteiras de 1–5, geradores determinísticos por semente e formato e rótulos preservados. Elas são inspiradas no benchmark CIFAR-10-C [22]; o ruído Gaussiano adiciona perturbações com desvios-padrão por severidade de 0,08–0,24; o frosted-glass blur permuta pixels em uma vizinhança 3×3 e combina-os com um borrão Gaussiano (força 0,2–1,0); o motion blur faz convolução com um kernel diagonal de largura 5–15; o fog combina a imagem com um mapa de ruído fractal de quatro oitavas.

B. Resultado da calibração

A calibração, executada uma vez a partir de um checkpoint limpo, selecionou as severidades da Tabela I. Para cada degradação, foram avaliadas as severidades de 1–5 e selecionou-se a menor com acurácia estritamente dentro de (0,25; 0,50) (procedimento na Seção IV-E).

C. Delineamento

O delineamento do estudo é o produto cartesiano das quatro degradações calibradas, cinco sementes (42–46) e três durações de rampa (5, 10, 20 passos), totalizando 60 episódios pareados com horizonte de operação fixo

Tabela II
Configuração fixa de cada episódio.

Parâmetro	Valor
Conjunto de dados	CIFAR-10, IID, 10 clientes
Treino / teste	50 000 / 10 000 imagens (5 000 por cliente)
Rodadas iniciais (limpas)	20
Passos de warm-up (limpos)	20
Duração do passo	10 s
Tamanho de lote / épocas	32 / 1
Modelo	Rede neural convolucional (CNN), 2,2M parâmetros (4 convolucionais, 2 densas)
Otimizador	Adam, taxa de aprendizado 10^{-3}
Velocidade / timeout	uniforme (percentil 75; rodada ≈ 8 s)
Detector	ADWIN $\delta = 0,002$, mínimo de 20 observações; MC dropout com $T = 5$
Quórum / janela	0,30 / 2 passos
Orçamento de retreinamento	1 rodada
Limiar de qualidade τ	0,50
Horizonte de operação	600 s
Sementes	42–46

Tabela III

Atraso de detecção (s) e progresso do drift no disparo (médias por grupo sobre 5 sementes; taxa de detecção por grupo; a semente Gaussiana que falhou em 100 s está excluída da média de atraso desse grupo).

Degradação	Atraso (s)			Progresso do drift no disparo 200 s	Taxa de detecção (de 5)
	50 s	100 s	200 s		
Fog	100	112	138	0,69	5/5
Frosted-glass	100	108	136	0,68	5/5
Motion blur	98	112	140	0,70	5/5
Ruído Gaussiano	164	158*	256	0,97	4/5*

Desvio-padrão do atraso entre sementes: ≤ 6 s para as degradações visuais; 71–104 s para o ruído Gaussiano. Para as rampas de 50 e 100 s o disparo ocorre após o fim da rampa, de modo que o progresso do drift no disparo é 1,00 por construção. A nota * indica média sobre as quatro sementes que detectaram.

de $H = 600$ s. Cada episódio emparea a política reativa com uma baseline que nunca retreina, no mesmo fluxo e checkpoint; as duas variantes compartilham o estado aleatório, o início da operação e o horizonte H , e a baseline registra, como contrafactual, o momento em que a política teria disparado. A Tabela II lista a configuração fixa.

VI. Resultados

Reportam-se médias por grupo sobre as cinco sementes e testam-se duas hipóteses: a política reativa reduz o downtime em relação à baseline sem retreinamento, e o atraso de detecção cresce com a duração da rampa. As medidas primárias são o atraso de detecção, o progresso do drift no disparo ($f(v_d)$, a fração da rampa percorrida no disparo) e o downtime; as secundárias são a retenção da acurácia limpa (acurácia limpa final menos a acurácia limpa antes do drift) e a acurácia final sob degradação.

A. Detecção: 59 de 60 episódios

A detecção teve êxito em 59 de 60 episódios (14 de 15 no cenário de ruído Gaussiano); a única falha ocorreu no ruído Gaussiano, na rampa de 100 s, com a semente 42. O atraso cresce com a duração da rampa para as três degradações visuais (Tabela III), em consonância com a análise do ADWIN para mudança gradual com inclinação constante [13]; o ruído Gaussiano não apresenta a mesma tendência.

B. Downtime: a variante reativa supera a baseline

A política reativa reduz o downtime médio nos 12 grupos (Tabela IV), de 408–562 s para 0–178 s, e apresenta menor

Tabela IV

Downtime (s): variante reativa versus baseline pareada sem retreinamento (médias por grupo sobre 5 sementes; formato: reativo / baseline).

Degradação	Duração da rampa		
	50 s	100 s	200 s
Fog	58 / 550	24 / 504	0 / 408
Frosted-glass	68 / 560	40 / 524	3 / 454
Motion blur	68 / 562	50 / 530	14 / 466
Ruído Gaussiano	130 / 558	178 / 526	112 / 448

A dispersão do ruído Gaussiano é grande (desvio-padrão do atraso de até 104 s; desvio-padrão do downtime de até 184 s, com downtime por semente entre 38 e 540 s); as degradações visuais têm desvio-padrão ≤ 6 s (atraso) e ≤ 10 s (downtime).

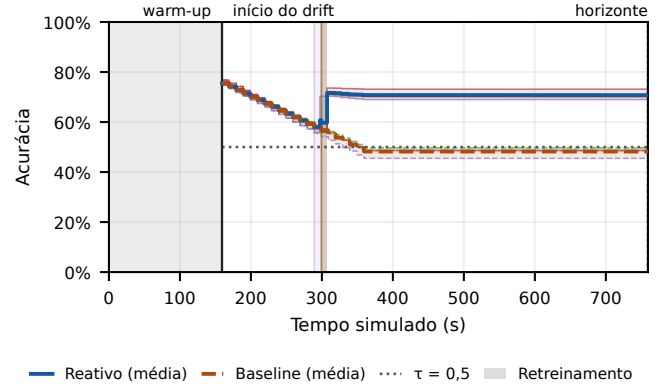


Fig. 1. Fog, rampa de 200 s, cinco sementes pareadas. Acurácia sob degradação ao longo do tempo simulado (sólida: reativo, tracejada: baseline; traços espessos: médias); $\tau = 0,50$ tracejada; faixas sombreadas: a rodada de retreinamento. A variante reativa permanece acima de τ (downtime nulo).

downtime em 59 dos 60 episódios pareados. O cenário de fog com rampa de 200 s atinge downtime nulo com $\tau = 0,50$ (Fig. 1); o disparo ocorre com o progresso do drift em 0,69, quando a acurácia ainda é de 0,57 (acima de τ); a rodada única mantém a acurácia acima de τ (acurácia final sob degradação de 0,71).

A ordenação entre a variante reativa e a baseline mantém-se ao variar o limiar de qualidade. Para $\tau \in \{0,40; 0,45; 0,50; 0,55\}$, o downtime da variante reativa não excede o da baseline pareada em nenhuma das 48 células, embora as magnitudes variem com τ (o contraste médio reduz-se em cerca de 47% com $\tau = 0,40$; o pior caso da variante reativa cresce para 190 s com $\tau = 0,55$).

C. Análise do quórum coletivo

Todos os 10 clientes emitem alarme em cada episódio detectado (59 de 59), uma vez que todos os clientes são degradados no episódio e cada detector opera sobre o próprio fluxo. O disparo ocorre no limiar do quórum (0,30, três clientes distintos na janela de dois passos) em 21 de 59 episódios; o ruído Gaussiano dispara no limiar em todos

Tabela V

Retenção da acurácia limpa Δ (variante reativa; média por grupo $\pm$ desvio-padrão sobre 5 sementes).

Degradação	Duração da rampa		
	50 s	100 s	200 s
Fog	$-0,096 \pm 0,012$	$-0,094 \pm 0,014$	$-0,014 \pm 0,005$
Frosted-glass	$-0,152 \pm 0,008$	$-0,156 \pm 0,014$	$-0,022 \pm 0,005$
Motion blur	$-0,199 \pm 0,017$	$-0,208 \pm 0,020$	$-0,016 \pm 0,002$
Ruído Gaussiano	$-0,084 \pm 0,013$	$-0,066 \pm 0,034$	$-0,072 \pm 0,016$

os episódios detectados da rampa de 100 s. A rampa longa dispersa os alarmes no tempo, e a fração sinalizada na decisão (a fração $|\hat{S}_{t_d}|/C$ no instante do disparo) diminui com a duração da rampa (de 0,88 para 0,42 em fog, de 0,90 para 0,56 em frosted-glass e de 0,66 para 0,54 em motion blur).

Sensibilidade a quórum e janela. Com quórum 0,50, a política dispara em 59 de 60 episódios, deixando de detectar a mesma semente de ruído Gaussiano que falha com o quórum padrão de 0,30, embora em rampa diferente. No quórum padrão, uma janela de um passo tem efeito neutro na média do conjunto (+3,7 s de atraso, -3,9 s de downtime), com a diferença variando de sinal entre os grupos; a união de dois passos não mostra benefício sistemático.

Staggered drift. Quando os clientes sofrem drift em passos diferentes (cliente i sofre drift no passo de operação i , sem rampa; 20 episódios), o gatilho coletivo dispara em 20 de 20 episódios no limiar do quórum (fração sinalizada 0,30–0,40). A política evita cerca de 81% do downtime da baseline (600 s, drift permanente) em três das quatro degradações, com 116 s de downtime na variante reativa, e 70% no ruído Gaussiano, com 180 s; como o drift é instantâneo e permanente, o atraso converte-se em downtime na mesma proporção (116 s correspondem a 108 s de atraso mais 8 s da rodada de retreinamento).

D. Taxa de falsos positivos em fluxos sem drift

Foram executados 15 episódios de controle (cinco sementes e três durações de rampa) sem degradação. Cada cliente monitora um fluxo limpo pelo horizonte completo de 600 s. O gatilho coletivo não dispara em nenhum episódio. Nenhum dos 15 episódios registra uma decisão de retreinamento, e os detectores locais emitem zero alarmes nas 9 000 avaliações cliente-passo. Ocorrem oito picos isolados de entropia (valores acima de 1,0), mas o ADWIN não sinaliza nenhum, pois o teste é de mudança na distribuição, não de picos. O gatilho, portanto, não dispara em fluxos sem drift e dispara sob drift (59 de 60 episódios; taxa de falsos positivos de 0%).

E. Retenção da acurácia limpa e comportamento de recuperação

Primeiro, rampas longas preservam a acurácia no conjunto limpo (Tabela V). Com rampa de 200 s, o conjunto de retreinamento mantém cerca de 30% de dados limpos

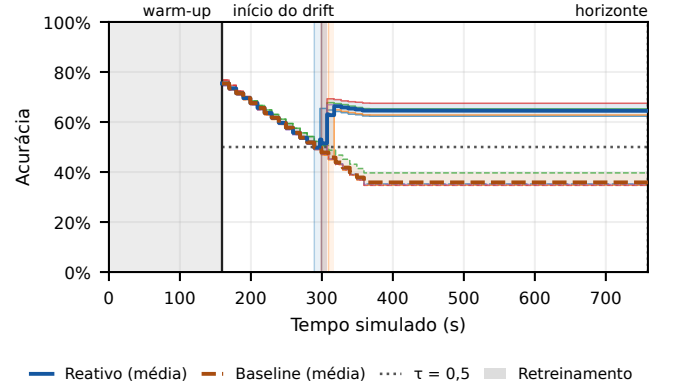


Fig. 2. Motion blur, rampa de 200 s, cinco sementes pareadas. A variante reativa recupera-se em uma rodada e permanece acima de τ durante a maior parte do horizonte; a baseline segue declinando até cerca de 0,35.

e a perda de acurácia limpa reduz-se a 1–2 pontos percentuais (ante 9–21 em rampas curtas) para fog, frosted-glass e motion blur; o ruído Gaussiano não apresenta esse ganho (progresso do drift de 0,97 no disparo). Segundo, a recuperação não é temporária, pois nenhum episódio volta a ficar abaixo de τ dentro do horizonte; o modelo retreinado generaliza para o regime totalmente degradado. Em 9 dos 20 episódios com rampa de 200 s (todos os cinco de fog, três de frosted-glass e um de motion blur), a acurácia permanece acima de τ . A acurácia final média por grupo, no regime totalmente degradado, varia de 0,60 a 0,71 para a variante reativa (exceto no episódio com falha, cuja acurácia final é 0,33) e de 0,35 a 0,48 para a baseline (Fig. 2).

VII. Discussão

O monitoramento também tem custo computacional. Ao longo do episódio de 80 passos (60 de operação mais 20 de warm-up), o detector com $T = 5$ custa cerca de 6,8 TFLOP (10^{12} operações de ponto flutuante; 4 000 forward passes em lote de 32, sendo 50 por passo nos dez clientes) ante cerca de 8,0 TFLOP da rodada única de retreinamento. A razão é de 0,85–0,96 dependendo da forma de contabilização, ou seja, o monitoramento responde por cerca de 47% da computação reativa. Nos dez clientes, o monitoramento por passo ocupa cerca de 0,13 s, 1,3% de cada passo de 10 s; no dispositivo medido, uma atualização do detector leva cerca de 12,7 ms por cliente. Caso a configuração publicada do UDD com $T = 50$ tivesse sido mantida, o monitoramento excederia a rodada de retreinamento em cerca de nove vezes, razão pela qual a adoção de $T = 5$ foi uma decisão de custo.

As conclusões sobre o downtime valem também para uma rodada dez vezes mais longa. Recalculando-se o downtime a posteriori a partir das séries registradas, atribuindo a acurácia pré-retreinamento ao intervalo $[v_d, v_d + d]$, em que d é a duração da rodada, usando o primeiro

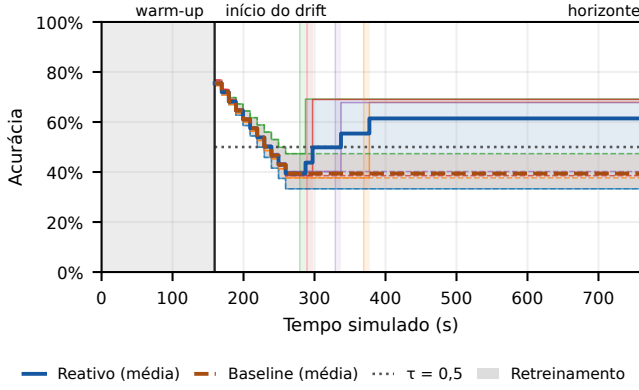


Fig. 3. Ruído Gaussiano, rampa de 100 s, cinco sementes pareadas. A semente 42 não dispara; sua trajetória (azul sólida) não se recupera, enquanto as outras quatro sementes se recuperam para cerca de 0,68.

registro pós-retreinamento em diante e aplicando a regra de orçamento, verifica-se que uma rodada de 80 s ainda reduz o downtime médio de 507,5 s (baseline) para 122,2 s (76%). A ordenação por grupo, o caso de downtime nulo no fog e o comportamento limitado pela detecção no ruído Gaussiano permanecem inalterados; o custo extra é exatamente o intervalo pré-retreinamento acrescido pela rodada mais longa. Com $d = 800$ s, a rodada já não cabe no horizonte de operação de 600 s e a regra de orçamento descartaria o retreinamento nos 59 episódios em que houve disparo, de modo que a política reativa passaria a equivaler à baseline por força da regra de orçamento, e não por dinâmica de treinamento; o limite é a menor diferença entre o instante de disparo e o horizonte, cerca de 140 s nesta configuração.

A. Ruído Gaussiano é o cenário limitante

O ruído Gaussiano é o único cenário com detecção incompleta. Com rampa de 100 s e semente 42, a política não dispara, pois somente quatro dos dez clientes emitem alarme em todo o episódio, jamais três dentro da mesma janela de dois passos (fração máxima sinalizada de 0,20 em passos esparsos); as outras quatro sementes detectam o drift em 120–210 s. Como a janela e o quórum são idênticos entre sementes, a falha é mais bem explicada pela perda de sensibilidade dos detectores locais. Sem decisão, a variante reativa equivale exatamente à baseline (downtime de 540 s). Com rampa de 200 s os atrasos variam entre 180 e 460 s (desvio-padrão de 104 s), mas a variante reativa ainda reduz o downtime em 75% em relação à baseline (112 vs. 448 s), em consonância com o risco que detectores do tipo ADWIN apresentam sob drifts longos e de transição suave [18]. O ruído Gaussiano é também o cenário com maior atraso de detecção. A Fig. 3 mostra a semente que falhou.

B. Por que a rampa longa apresenta melhores resultados

Para as degradações visuais, a rampa de 200 s tem três efeitos favoráveis. (i) O disparo ocorre no meio da rampa,

com progresso do drift de 0,68–0,70, em que a acurácia na decisão é 0,57 para fog (ainda acima de τ), 0,51 para frosted-glass e 0,48 para motion blur. (ii) O conjunto de retreinamento contém cerca de 30% de dados limpos, reduzindo o esquecimento catastrófico. (iii) O modelo retreinado com progresso do drift de $\approx 0,7$ generaliza para o regime totalmente degradado, de modo que a recuperação se sustenta. Nas rampas curtas, a detecção termina após a rampa e o conjunto de retreinamento torna-se quase totalmente degradado, como no regime abrupto. O ruído Gaussiano dispara mais tarde (progresso 0,97) e não apresenta os ganhos (i) e (ii).

C. Limitações

Os resultados limitam-se às seguintes restrições declaradas. (i) Cinco sementes por grupo limitam a precisão da estimativa de dispersão. (ii) O modelo é uma CNN pequena de cerca de 2,2 milhões de parâmetros; as acurácias absolutas sob degradação são modestas, como esperado para arquiteturas pequenas [22]. (iii) O cenário pressupõe um único drift, uma rodada de retreinamento, partição IID, velocidade uniforme e horizonte fixo. (iv) O progresso do drift no disparo é informativo somente na rampa de 200 s. (v) O UDD publicado usa $T = 50$ forward passes; o $T = 5$ adotado aqui reduz o orçamento de monitoramento por uma questão de custo, e a comparação direta entre as duas configurações não foi investigada aqui, permanecendo como trabalho futuro; $\delta = 0,002$ é o valor padrão da implementação de referência, e não um valor calibrado [11], [18]. (vi) O downtime é amostrado na granularidade do passo mais o instante de conclusão da rodada. (vii) A proteção contra falsos positivos que o quórum oferece não pôde ser quantificada, de modo que as afirmações deste trabalho restringem-se à sua viabilidade e à taxa nula de falsos positivos em fluxos sem drift (0/15 episódios de controle). Comparadores com agendamento, do tipo oráculo e com detector centralizado permanecem como trabalhos futuros. (viii) Quinze episódios de controle podem subestimar a taxa de falsos positivos do detector sob monitoramento contínuo.

VIII. Conclusão e Trabalhos Futuros

Este trabalho propõe e caracteriza uma política de decisão coletiva por quórum para o retreinamento reativo em aprendizado federado síncrono sob drift gradual. Cada cliente monitora seu fluxo sem rótulos, e o servidor somente dispara o retreinamento quando a confirmação de clientes distintos atinge o quórum em uma janela de passos. A política foi avaliada contra uma baseline pareada sem retreinamento em 60 episódios, com o downtime de serviço como métrica primária. A detecção teve êxito em 59 de 60 episódios, e o atraso cresce com a duração da rampa para as degradações visuais; a variante reativa reduziu o downtime médio de 408–562 s para 0–178 s (zero no caso de fog com rampa de 200 s); e rampas longas preservaram a acurácia limpa para as degradações

visuais. O gatilho coletivo dispara nos 20 episódios de staggered drift (70–81% do downtime da baseline evitado) com taxa de falsos positivos de 0% em 15 controles sem drift. Trabalhos futuros incluem baselines com detector centralizado, do tipo oráculo e com agendamento; mais sementes e análise formal de dispersão; drift recorrente; partições não IID; drift incremental; modelos maiores; sensibilidade aos parâmetros T , δ e limites de calibração do detector.

Disponibilidade

A implementação e todos os artefatos (os 60 episódios pareados, variantes comparadoras, resumos agregados e tabelas por semente) estão disponíveis em <https://github.com/julio-amaral-oliveira/FederatedLearningSimulation>.

Referências

- [1] B. McMahan, E. Moore, D. Ramage, S. Hampson, and B. Agüera y Arcas, “Communication-efficient learning of deep networks from decentralized data,” in *Proc. AISTATS, PMLR* 54, pp. 1273–1282, 2017.
- [2] F. E. Casado, D. Lema, M. F. Criado, R. Iglesias, C. V. Regueiro, and S. Barro, “Concept drift detection and adaptation for federated and continual learning,” *Multimedia Tools and Applications*, vol. 81, pp. 3397–3419, 2022.
- [3] V. Cerqueira, H. M. Gomes, M. Heyden, B. Pfahringer, and A. Bifet, “A framework for evaluating and benchmarking concept drift detection methods,” in *Proc. KDD*, 2026, doi: 10.1145/3770855.3819070.
- [4] E. Jothimurugesan, K. Hsieh, J. Wang, G. Joshi, and P. B. Gibbons, “Federated learning under distributed concept drift,” in *Proc. AISTATS, PMLR* 206, pp. 5834–5853, 2023.
- [5] T. Chow, U. Raza, I. Mavromatis, and A. Khan, “FLARE: Detection and mitigation of concept drift for federated learning based IoT deployments,” in *Proc. IEEE IWCNC*, pp. 989–995, 2023.
- [6] I. Mavromatis, S. De Feo, and A. Khan, “FLAME: Adaptive and reactive concept drift mitigation for federated learning deployments,” *arXiv:2410.01386*, 2024.
- [7] A. Patel, D. S. Tomar, R. K. Pateriya, and R. Haripriya, “FL-MalDrift: A federated learning framework for malware detection under local concept drift,” *Scientific Reports*, vol. 16, Art. 1821, 2026.
- [8] Y. Han, D. Wu, and B. Varghese, “DriftGuard: Mitigating asynchronous data drift in federated learning,” *arXiv:2603.18872*, 2026.
- [9] P. Fu and M. Tang, “FedDAA: Dynamic client clustering for concept drift adaptation in federated learning,” *arXiv:2506.21054*, 2025.
- [10] Q. Zhou et al., “FedPLC: Federated learning with dynamic cluster adaptation for concept drift on non-IID data,” *Sensors*, vol. 26, no. 1, Art. 283, 2026.
- [11] L. Baier, T. Schlör, J. Schöffner, and N. Kühl, “Detecting concept drift with neural network model uncertainty,” in *Proc. 56th Hawaii Int. Conf. on System Sciences (HICSS)*, pp. 835–844, 2023.
- [12] Y. Gal and Z. Ghahramani, “Dropout as a Bayesian approximation: Representing model uncertainty in deep learning,” in *Proc. ICML, PMLR* 48, pp. 1050–1059, 2016.
- [13] A. Bifet and R. Gavaldà, “Learning from time-changing data with adaptive windowing,” in *Proc. SIAM Int. Conf. on Data Mining*, pp. 443–448, 2007.
- [14] J. L. Lobo, I. Laña, E. Osaba, and J. Del Ser, “On the connection between concept drift and uncertainty in industrial artificial intelligence,” in *Proc. IEEE CAI*, pp. 171–172, 2023.
- [15] Y. Ovadia et al., “Can you trust your model’s uncertainty? Evaluating predictive uncertainty under dataset shift,” in *Proc. NeurIPS*, pp. 13991–14002, 2019.
- [16] V. Nguyen, C. Shui, V. Giri, S. Arya, A. Verma, F. Razak, and R. G. Krishnan, “Reliably detecting model failures in deployment without labels (D3M),” in *Proc. NeurIPS*, 2025.
- [17] O. Solozobov, “Label-free detection of governance evidence degradation in risk decision systems,” *arXiv:2604.17836*, 2026.
- [18] L. Poenaru-Olaru, L. Cruz, A. van Deursen, and J. S. Reller-meyer, “Are concept drift detectors reliable alarming systems?” *arXiv:2211.13098*, 2022.
- [19] O. A. Mahdi, E. Pardede, S. Bevinakoppa, and N. Ali, “Federated learning under concept drift: A systematic survey of foundations, innovations, and future research directions,” *Electronics*, vol. 14, no. 22, Art. 4480, 2025.
- [20] M. Böther et al., “Modyn: Data-centric machine learning pipeline orchestration,” *Proc. ACM on Management of Data*, vol. 3, no. 1, Art. 55, 2025.
- [21] M. F. Criado, F. E. Casado, R. Iglesias, C. V. Regueiro, and S. Barro, “Non-IID data and continual learning processes in federated learning: A long road ahead,” *Information Fusion*, vol. 88, pp. 263–280, 2022.
- [22] D. Hendrycks and T. Dietterich, “Benchmarking neural network robustness to common corruptions and perturbations,” in *Proc. ICLR*, 2019.
- [23] K. Panchal et al., “Flash: Concept drift adaptation in federated learning,” in *Proc. ICML, PMLR* 202, pp. 26931–26962, 2023.
- [24] X. Chen et al., “Virtual concept drift detection and adaptation in federated data stream learning,” *Int. J. Data Science and Analytics*, vol. 21, Art. 46, 2026.
- [25] M. Stallmann, A. Wilbik, and G. Weiß, “Towards unsupervised sudden data drift detection in federated learning with fuzzy clustering,” in *Proc. IEEE FUZZ-IEEE*, 2024.
- [26] R. A. Bhope, K. R. Jayaram, P. Venkateswaran, and N. Venkatasubramanian, “Shift happens: Mixture of experts based continual adaptation in federated learning,” in *Proc. ACM Middleware*, pp. 455–468, 2025.
- [27] G. Canonaco, A. Bergamasco, A. Mongelluzzo, and M. Roveri, “Adaptive federated learning in presence of concept drift,” in *Proc. IEEE Int. Joint Conf. on Neural Networks (IJCNN)*, pp. 1–7, 2021.
- [28] P. Lu, J. Lu, A. Liu, and G. Zhang, “Early concept drift detection via prediction uncertainty,” in *Proc. AAAI Conf. on Artificial Intelligence*, vol. 39, no. 18, pp. 19124–19132, 2025.
- [29] B. Zhang, H. Guan, H. K. Lee, R. Liu, J. Zou, and L. Xiong, “FedSGT: Exact federated unlearning via sequential group-based training,” *arXiv:2511.23393*, 2025.